2. GENETIKUS ALGORITMUSOK ALAPJAI

- Comparing to the traditional search and optimization methods, genetic algorithms are robust, global. (Finding solution with probability 1)
- Global stochastic search method.
- Slow, usually not suitable for on-line problem solution
- Ideas are adopted from nature (Evolution)
- Operates on populations with genetic operators. Population is improved to adapt to the environment. The solution candidate is the best individual of the population.

2.1. Basic notions

Gene:

- Material carrying genetic information, more or less permanent organization unit of the dezoxiribonucleinacid (DNA).
- **determines the appearance of certain properties or property groups**
- Managing to get unchanged in every successor cell or successor organism by repeated self-duplication, implements genetic continuity in the cell generations and generation series following each other.

Chromosome:

- Carrier of the inheriting material, the genes, which is organized into longer or shorter spring-like bodies
- Chromosomes are embedded in the plasma of the cell center.
- They keep their individual properties throughout the whole life cycle of the cell, but their material concentrates only at the fission of the cell as much as they are visible for microscopic observation.
- **Between shape-identically constructed chromosome pairs, in a certain life period of the cell (in the beginning of the meiosis), an odd attraction occurs. Individual members of the chromosome pairs fit lengthways, and single or multiple crossover happens. The paired chromosomes later separate again. The originated chromosomes usually resemble the ones made connection.**

Allel:

- One of the structurally and functionally altered versions of a gene. Different alternative allels of the same gene come into existence by mutation from each other, and are formed into each other the same way.
- **A gene is represented by one of its allels at a place in the chromosome.**
- In natural populations the most frequent form is usually called original or wild type.
- A mutant allel arisen by altering some point of the gene, compared to the wild allel, can be of variant strength: unable to display visible effect, displaying weaker or stronger visible effect than the wild type, opposite effect to the wild or qualitatively different effect from the wild type

Genotype (≈*"code"*):

- **Sum of the genetic information stored in the chromosomal genes of the organism.**

Fenotype:

- **Sum of the perceptible, determinable (describable and measurable) inner and outer properties, resultant of the inheritable base and the life conditions**

Individual:

- Organizational, physiological and reproduction-biological (genetic) unit of the living world, in other names living being or organism
- separates from its environment, exercises metabolism, propagates.
- Sum of individuals of identical origin, arisen via the genital process, is the population, while individuals arisen via non-genital way can not be considered as independent individuals until they are connected with their parent.
- Individuals in the nature usually make communities (supraindividual organizations) and fill the biosphere forming populations, associations and ecological systems

Population:

- Organizational unit above the individual level, which means sum of individuals sharing the same quality of a certain peculiarity, feature within the species
- **According to the genetic interpretation population is a smaller or larger group of living beings belonging to the same species, in which the possibility of propagation between individuals exists.**
- Every species exists in a form of populations.
- Due to the adaptation to the environment, the populations forming species more or less differ from each other regarding their phenotype and genotype.
- The ideal population consists of a large number of individuals; the number of individuals remains constant through generations, because it is not subject to the natural selection caused by the environment

Species:

- Fundamental unit of the systematization of the living beings and the evolution, which usually consists of populations of living beings similar to each other.
- By permanent or occasional exchange of the genes between the individuals of these populations, continuous variation of several properties arise.
- **The emphasis of the biological species concept falls to the possibility of gene exchange**

Evolution:

- Production of living material from lifeless (biogenesis) and development of diversity
- Evolution is unequal in space and time, continuous and irreversible process.
- **During evolution organism arise, which can balance disadvantageous environmental effects.**
- Evolution genetics: mutation, selection, isolation, recombination.

Selection:

- **Naturally or artificially initiated biological process, which hinders the survival or propagation of certain individuals or groups, while doesn't hinder others.**
- non-random propagation of different phenotypes
- Selection is much more effective in large population, while smaller populations are dominated rather by genetic drift
-

Recombination (crossover)

- **Exchange between genetic materials containing different genotypes, whose result is the recombined genotype of the descendant, differing from both parents**
- development of one or more new combination of genes from two, partly different parental gene sets..

Mutation:

- **Change in succession materials, which is not a result of genetic recombination.**
- A mutant is an individual in which by means of mutation at least one gene locus has changed.
- A characteristic property of mutants is that in morphological nature (height, number of leaves, spike shape) can be well distinguished.
- Mutations have important role in development of new species.
- The rate of spontaneous gene mutations is extremely small.

Migration:

- **Spreading of plant and animal species or their totality, flora and fauna.**

Fitness (alkalmassági érték):

- **Upon given external conditions the chance of an individual or individuals, incarnating a given genotype, for staying alive and giving birth such viable descendants which will take part in forming the different generations.**

Relation between genetic Algorithm (GA) and genetics

Genetic algorithms use ideas and rules taken from natural genetics with significant simplifications in the global stochastic optimum searching procedure.
They primarily employ the following principles:

- Let there be a population of individuals. Every individual is a string over an alphabet. The individuals are different
- Let there be genetic operations, which alter the individuals
- Let there be a function, which defines a fitness (competence) value for every individual
- After multiple changes the rearrangement of the population occurs based on the fitness value (reproduction)
- The reproduction, which leads to the survival of chromosomes having high fitness value and dying out of those having smaller value, finally acts so that from generation to generation the chromosomes, from the point of view of the problem to be solved, become better

Similar approaches:

- Evolutionary algorithms (only selection and mutation)
- Genetic programming (generalization). If the original parameter-dependent system which should be optimized as a function of these parameters, is substituted with a theoretical construction such as a computational rule or a computer program, then the rule or program which optimally solves the formulated problem can be searched for.

2.2. Principles of genetic algorithms

2.2.1. Concept

Although genetic algorithms can be used in case of multi-criterion (multiobjective) optimization problems, hereinafter only the optimization of single objective function is considered

Problem: the calculation of the global minimum of the unconstrained real function $f\left(x_1,...,x_{N\,\mathrm{var}}\right)$:

$$\min f\left(x_1,...,x_{N\,\mathrm{var}}\right)$$

Implementation: MATLAB

2.2.2. Structure of SGA (Single Genetic Algorithm)

```
NIND=40;                    %Number of individuals
MAXGEN=300;                 %Maximum no. of
                               % generations
NVAR=20;                    %No. of variables
PRECI=20;                   %Precision of variables
GGAP=0.9;                   %Generation map

%Build field descriptor
FieldD=[rep([PRECI],[1,NVAR]);rep([-
512;512],[1,NVAR]);... rep([1;0;1;1],[1,NVAR])];

%Initialize population. crtbp:create binary population
Chrom=crtbp(NIND,NVAR*PRECI);

gen=0;                      %Counter

%Evaluate initial population. bsrv: binary string to real
% value
ObjV=objfun1(bs2rv(Chrom,FieldD));
```

```
%Generational loop

while gen<MAXGEN,
      %Assign fitness values to entire population
      FitnV=ranking(ObjV);

      %Select individuals for breeding
      SelCh=select('sus',Chrom,FitnV,GGAP);

      %Recombine individuals (crossover)
      Selch=recombin('xovsp',Selch,0.7);

      %Apply mutation
      SelCh=mut(SelCh);

      %Evaluate offspring, call objective function
      ObjVSel=objfun1(bs2rv(SelCh,FieldD));

      %Reinsert offspring into population
      [Chrom ObjV]=
            reins(Chrom,SelCh,1,1,ObjV,ObjVSel);

      %Increment counter
      gen=gen+1;

end
```

- FieldD: Describes the properties of variables (Allels in Chromosome) [back to algorithm]

	1	2	…	NVAR
1.	20 [PRECI]	20 [PRECI]	…	20 [PRECI]
2.	-512 [also korlát]	-512 [also korlát]	…	-512 [also korlát]
3.	512 [felső korlát]	512 [felső korlát]	…	512 [felső korlát]
4.	1 [kód=1-Gray, 2-bináris]	1 [kód=1-Gray, 2-bináris]	…	1 [kód=1-Gray, 2-bináris]
5.	0 [skála= 1-log, 0-lin]	0 [skála= 1-log, 0-lin]	…	0 [skála= 1-log, 0-lin]
6.	1 [alsó korlát is felvehető]	1 [alsó korlát is felvehető]	…	1 [alsó korlát is felvehető]
7.	1 [felső korlát is felveh.]	1 [felső korlát is felveh.]	…	1 [felső korlát is felveh.]

- Chrom – The chromosomes of individuals of population in separate rows.
  Numbers in cells are set randomly based on FieldD:

	Variable #1	Variable #2	…	Variable #NVAR
Individial #1	20 bit	20 bit	…	20 bit
Individial #2	20 bit	20 bit	…	20 bit
Individial #NIND	20 bit	20 bit		20 bit

]

Inicialization and decoding population:

crtbp - create binary population
crtrp - create real population
bs2rv - binary string to real value

- Az $\mathbf{x} = (x_1, ..., x_{N_{\text{var}}}) \mapsto f(\mathbf{x})$ objfun1 resturns with the cost of individuals in column vector ObjV

[back to algorithm]

➢ *2.2.3. MPGA felépítése*

```
%Define GA Parameters
GGAP=0.8;          %Generation gap
XOVR=1;            %Crossover rate
MUTR=1/NVAR;       %Mutation rate
MAXGEN=1220;       %Maximum no. of
generations
INSR=0.9;          %Insertion rate
SUBPOP=8;          %No. of subpopulations
MIGR=0.2;          %Migration rate
MIGGEN=20;         %No. of
generations/migration
NIND=20;           %No. of
individuals/subpop
%Specify other funtions as strings
SEL_F='sus';       %Name of the selection
funtion
XOV_F='recdis';    %Name of the
recombination funtion
MUT_F='mutbga';    %Name of the mutation
function
OBJ_F='objharv';   %Name of objective
function
Chrom=crtrp(SUBPOP*NIND,FieldD);
gen=0;
ObjV=feval(OBJ_F,Chrom);
```

```
%Generational loop

while gen<MAXGEN,
  %Fitness assignment to whole population
       FitnV=ranking(ObjV,2,SUBPOP);
  %Select individuals from population
      SelCh=select(SEL_F,Chrom,FitnV,GGAP,SUBPOP);
  %Recombine selected individuals
      SelCh=recombin(XOV_F,SelCh,XOVR,SUBPOP);
  %Mutate offspring
      SelCh=mutate(MUT_F,SelCh,FieldD,[MUTR],SUBPOP);
  %Calculate objective function for offsprings
      ObjVOff=feval(OBJ_F,SelCh);
  %Insert best offspring replacing worst parents
      [Chrom,ObjV]=reins(Chrom,SelCh,SUBPOP,...
                                [1 INSR],ObjV,ObjVOff);
  %Increment counter
      gen=gen+1;
  %Migrate individuals between subpopulations
      if (rem(gen,MIGGEN)==0)
        [Chrom,ObjV]=
                migrate(Chrom,SUBPOP,[MIGR,1,1],ObjV);
      end
end
```

2.2.4 Fitness functions

[back to algorithm]

Given: The objective function (scalar-valued criterion) is a $\mathbf{x} = \left(x_1,\ldots,x_{N_{\text{var}}}\right) \mapsto f(\mathbf{x})$ mapping.

Expected: Fitness function $F(\mathbf{x}) = g(f(\mathbf{x}))$ should preserve order and be positive

The ranking function can order fitness (competency) values to the values of Nind objective function based on specified principles and place them in the *FitnV* vector

Let the individuals of the population in phenotype (real) form be the $\mathbf{x}_i$ vectors: $\{\mathbf{x}_i\}_{i=1}^{N_{ind}}$.

Principles:

- In case of positive objective function the proportional fitness is defined by

$$F(\mathbf{x}_i) = \frac{f(\mathbf{x}_i)}{\sum_{i=1}^{N_{ind}} f(\mathbf{x}_i)}$$

[back to algorithm]

- In case of scaling**:**

Code: FitnV=scaling(ObjV,Smult);

Procedure: $F(\mathbf{x}_i) = af(\mathbf{x}_i) + b$

where *a*, *b* are the scaling factor and offset, respectively. The scaling function automatically determines the values of *a* and *b* from the input variable S_{mult} so that if the average value of *ObjV* is f_{ave}, then the upper bound of the maximal fitness is $f_{ave} * S_{mult}$. If one of the values of *ObjV* is negative, then the scaling tries to find an appropriate *b* value so that the fitness becomes positive. Since it is not always possible, using of scaling should be avoided if $f(\mathbf{x}) < 0$

[back to algorithm]

➢ *Ranking*

Code: FitnV=ranking(ObjV);

Procedure: $f(\mathbf{x}_i)$ values are sorted so that in the $f(\mathbf{x}_i) \rightarrow pos$ mapping

$$\max f(\mathbf{x}_i) \rightarrow pos(f(\mathbf{x}_i)) = 1, \ldots, \min f(\mathbf{x}_i) \rightarrow pos(f(\mathbf{x}_i)) = N_{ind}$$

is satisfied

❖ Linear

$$F(\mathbf{x}_i) = 2 - sp + 2 * (sp - 1)\frac{pos(f(\mathbf{x}_i)) - 1}{N_{ind} - 1}$$

keeping the order, *2-sp* is assigned to the maximum of the objective function values and *sp* to the minimum (e.g. in case of *sp=2* the most competent individual gets the fitness value 2, and the least competent gets 0)

❖ Nonlinear

1. Determine the roots of

$$(sp - N_{ind})x^{N_{ind}-1} + sp * x^{N_{ind}-2} + \ldots + sp * x + sp = 0$$

2. Determine the fitness values

$$F(\mathbf{x}_i) = \frac{N_{ind} * X^{pos(f(\mathbf{x}_i))-1}}{\sum_{i=1}^{N_{ind}} X^{i-1}}$$

This method assigns *sp* to the minimal objective function value while to the others it assigns nonlinearly descending values, keeping the order.

The reason of assigning the fitness to the ranking position instead of the value of the objective function is used to forbid early convergence to a local minimum.

[back to algorithm]

2.2.5. Selection methods

[vissza algoritmushoz]

Selection is the determination of the number of occasions of an individual is selected for recombination (crossover); that is, the number of occasions it takes part in making a descendant (offspring).

Selection consists of two independent processes

1. determination of occasions for an individual wich it can count on
   (Fitness $\rightarrow$ *the probability of taking part in the crossover*)

2. converting the expected number of occasions to a discrete number of individuals.
   (*the probability of taking part in the crossover* $\rightarrow$ The actual number of individuals designated to crossover)

Properties of selection process:

- **Bias:** the difference between the actual and expected selection probability $\rightarrow$ *Precision*
- **Spread:** the value interval of the possible experiences, in which an individual can take part $\rightarrow$ *Consistency*

Goal for efficiency: zero bias, minimal spread and little time

Most popular methods:

- Roulette wheel selection $\rightarrow$ complexity: $\sigma(N \log N)$
- Stochastic universal sampling $\rightarrow$ complexity: $\sigma(N)$

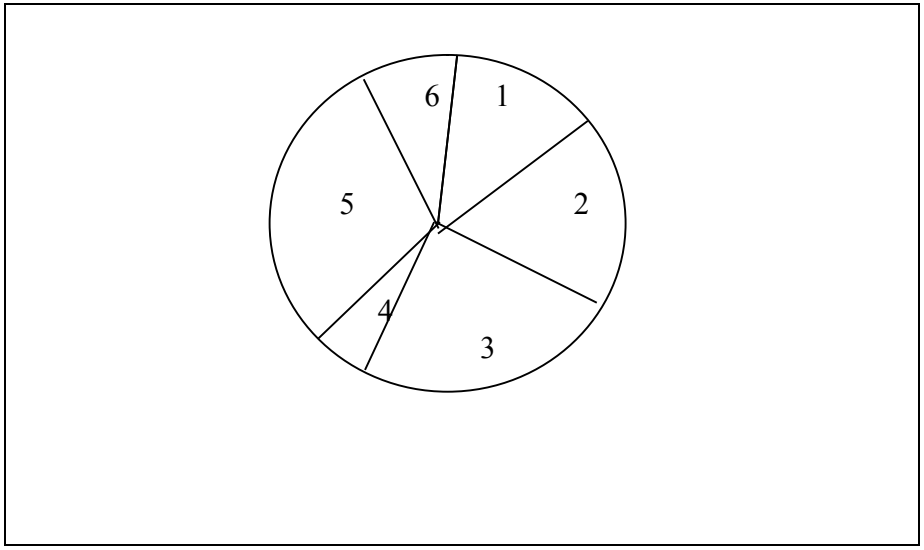
- *Roulette wheel selection*

1. A real valued interval *Sum* is defined, which can be the sum of the expected selection probability of the individuals or the sum of the fitness values of the individuals in the population

2. The individuals can be mutually unambiguously projected to contiguous intervals within the interval [*0,Sum*]. Sizes of the intervals of the individuals correspond with their fitness values.

3. Individual selection by generating a uniformly distributed random number on the interval [*0,Sum*]

4. The process is repeated until the necessary number of individuals is selected.



Variations:

- Stochastic sampling with replacement – SSR

The sizes of the segments and the selection probability remains the same during the whole selection phase (see above)

- Stochastic sampling with partial replacement – SSPR

Each time an individual is selected, the value of its segment is reduced by 1 while it is positive, then sticks to zero

- upper limit for spread
- lower limit is zero
- bias is bigger than in the SSR

The following methods have two phases:

- In the integer phase the individuals are selected deterministically by the integer part of the expected value of the experiments
- The remaining individuals are selected randomly by the fractional part of the expected value.

- Remainder stochastic sampling with replacement - RSSR
  The roulette-wheel selection for the individuals use selection non-deterministically. During the roulette-wheel phase the fractional part remains unchanged, thus they stay in race.

  - lower limit for spread
  - Zero bias.

- Remainder stochastic sampling without replacement - RSSWR
  It substitutes the fractional part of the expected value of the individual with zero, if it was selected during the fraction phase.

  - Minimal spread
  - Bias appears (since it prefers smaller fractional parts).

- *Stochastic universal sampling - SUS*

Unlike the roulette-wheel selection, when one selection pointer was used, the SUS method uses N equidistant pointers, where N is the number of selections.

1. Shuffle the population
2. generating random number in intervak [0, *Sum/N*].
3. Selection of N individual by pointers $\{ptr, ptr + \frac{sum}{N}, \ldots, ptr + (N-1)\frac{sum}{N}\}$.

Properties:

- Minimal spread
- Zero bias $\rightarrow$

THE BEST !!!

Code: SelCh=select('sus',Chrom,FitnV,GGAP);

'sus' - Stochastic universal sampling
'rws' - Roulette wheel selection

[vissza algoritmushoz]

2.2.6. Recombination

[vissza algoritmushoz]

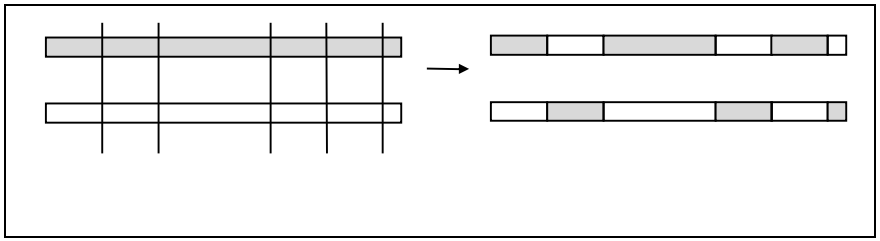
- Method to generate new chromosomes.
- Follows the Selection. Selected individuals are organized (randomly) into pairs for crossover.
- The reinsertation into population after crossover can be carried out
  - randomly
  - based on their fittness

- *Recombination in the case of binary population*

  Let L be the length of the chromosome string

  - Single point crossover
    1. generate a uniform random number i in interval $[1, L-1]$ for every pair.
    2. Change chromosome parts from bit i

  - Multipoint crossover
    1. M crossover points randomly chosen: $k_i \in \{1,2,...,L-1\}$
    2. Exchange every even parts in the Chromosomes.



  - Uniform crossover: generalizes the previous scheme so that every chromosome place can become crossover point. This can be implemented by generating a mask of length L.

The motivation of multipoint crossovers:

Those parts of the chromosome, which affect the quality index (objective function value) significantly into good direction, usually follow each other in non-connecting substrings.

Conclusions:

→ Multipoint crossover prevents convergence towards a big fitness individual in the early stage of the algorithm, thus prohibits sticking to a local minimum.

→ makes the method robust

Special crossover operators**:**

- Shuffle:
  1. Shuffles the bits before recombination
  2. one-point crossover
  3. Unshuffle the bits

- Reduced surrogate operator
  enforces descendants altering from the parents
  → It tests the selected crossover places, and accepts only if the genes of the parents differ at the crossover places.

- *Recombination in the case of binary population*

  - binary recombination operators are not applicable.

  - The new phenotype values, in case of real population, arise from and between the phenotype values of the odd/even parents

Variations:

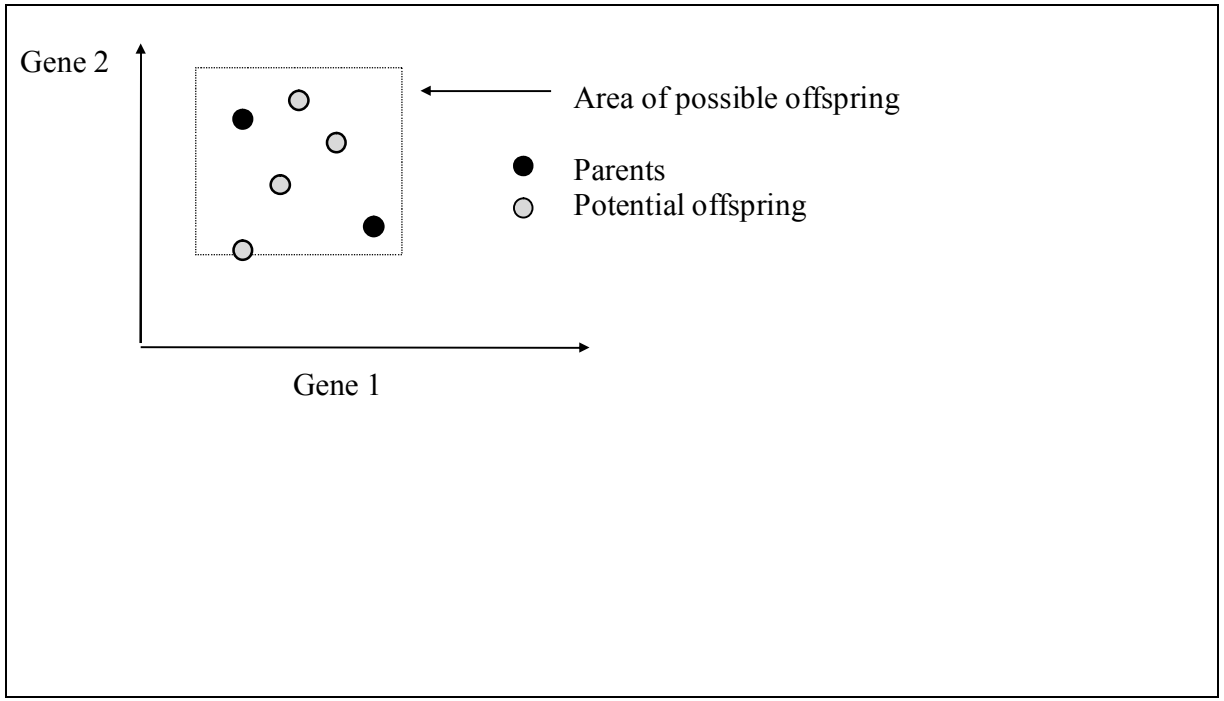
- intermediate recombination

- line recombination

➢ Intermediate recombination

The phenotype values O_1 or O_2 of the offspring arise from the phenotype values of the parents P_1 and P_2 with respect to a factor **a**, which is a random vector. Its components are selected from the $[-0.25,1.25]$ interval

$$\sigma = P_1 + \mathbf{a}. * (P_2 - P_1) \qquad \dim \mathbf{a} = N_{\text{var}}$$

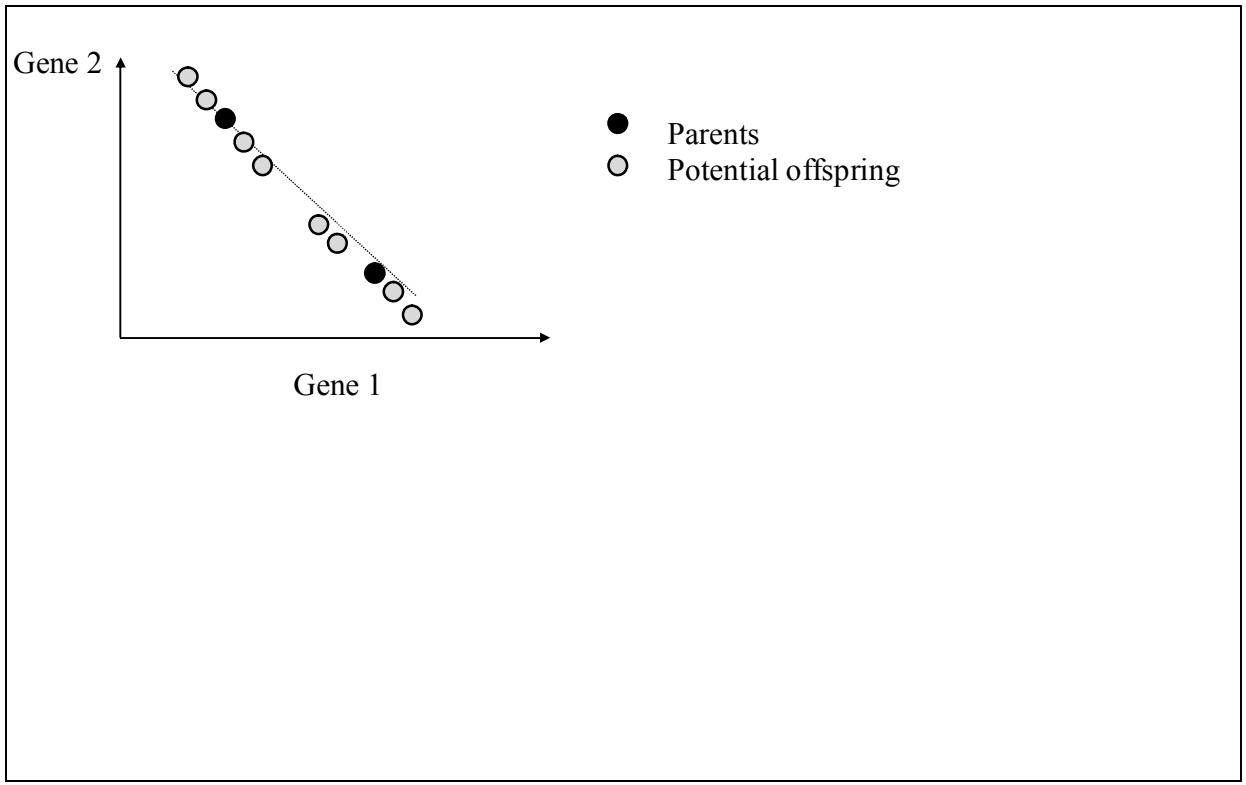


- Line recombination

$$\sigma = P_1 + \alpha * (P_2 - P_1)$$

where α is a random number.

It is a degenerating case of the intermediate recombination, where all components of the vector **a** are equal.



Remarks:

- Binary recombination operators on real populations would be senseless, because the resulting genetic material in the recombination would significantly differ from the values of the decision variables (x_i) of the parents, which contradicts the expectation of the offspring containing genes from both parents.

- **Code:** Selch=recombin('xovsp',Selch,0.7);

'xovsp'	-	Single point crossover (bináris)
'xovmp'	-	General multi point crossover (bináris)
'xovsh'	-	Shuffle crossover (bináris)
'reclin'	-	line recombination (real)
'recint'	-	intermediate recombination

[vissza algoritmushoz]

2.2.7. Mutation

[vissza algoritmushoz]

- random process, in which an allel of the gene is substituted to create a new genetic structure.
- low probability rate; its typical value is between 0.001 and 0.01.
- Goal: ensure that the search probability of every string is not zero and the good genetic information is not lost during selection and crossover.
- **Code:** SelCh=mut(SelCh);

- *Mutation on binary population:*

Illustration:

Mutation point ↓											binary	Gray
Original string-	0	0	0	1	1	0	0	0	1	0	0.9659	0.6634
Mutated string-	0	0	1	1	1	0	0	0	1	0	2.2146	1.8439

Mutating a single bit (depending its position) can cause significant changes in the value of the phenotype in both coding

Gray code can be advantageously used in genetic algorithms, because the stochastic search is less likely to be deceived by the regular Hamming distance between the quantizing intervals

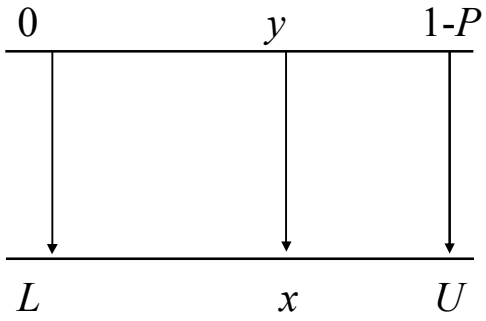
Scaling:

Let *n=len* be the bit number and let $P = \left(\frac{1}{2}\right)^n$ be the precision . The binary string $(b_1, b_2, ..., b_n)$ corresponds to the fixed point number

$$y = \sum_{i=1}^{n} b_i \left(\frac{1}{2}\right)^i$$

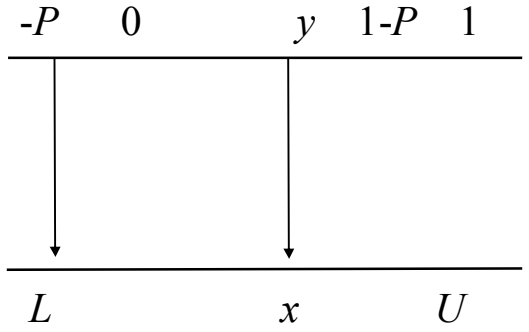
The number is in the interval [0,1-P]

Bounds are not excluded:



$$x = \frac{y}{1-P}(U - L) + L$$

Bounds are excluded:



$$x = \frac{y+P}{1+P}(U - L) + L$$

2.2.7.2. Mutation on real population:

Implementation:

- Perturbation of genes or
- Random selection of the new values

$$x_{mut} = x + Mutmask * \frac{U - L}{2} * Mutshrink * Delta$$

Mutshrink: Shrinking parameter
Mutmask: Mask selecting a variable with *Muterate* probability for mutation

Its value: -1,0 vagy 1

Delta $Delta = \sum_{i=1}^{m-1} \alpha_i 2^{-i}$,

where m is the bit number and α_i becomes *1* with *1/m* probability, otherwise zero

In case of $m = 20$ the selected mutation operator is able to find the place of the optimum with $0.5(U - L)Mutshrink \cdot 2^{-19}$ precision

[vissza algoritmushoz]

2.2.8. Reinsertation

[vissza algoritmushoz]

After the new generation was created from the old population by selection, recombination and mutation, the objective values of the individuals of the new population have to be determined.

If less individuals were born during recombination, then the difference between the number of individuals in the new and old population is defined as **generation gap**.

- If only a few new individuals came into existence → steady-state or incremental genetic algorithm.
- If the best individuals are deterministically allowed to live from generation to generation → elitist strategy

Selection of the individuals for reinsertion can be executed

- Randomly (uniform selection)
- Based on their fitness

The reinsertion rate of the individuals can be given proportionally to the size of the population, which indirectly determines the number of elite individuals.

Code: reins

[vissza algoritmushoz]

2.2.9. Migration

[vissza algoritmushoz]

- In case of multipopulation the population consists of subpopulations
- The new subpopulation will always be reinserted to the corresponding old subpopulation

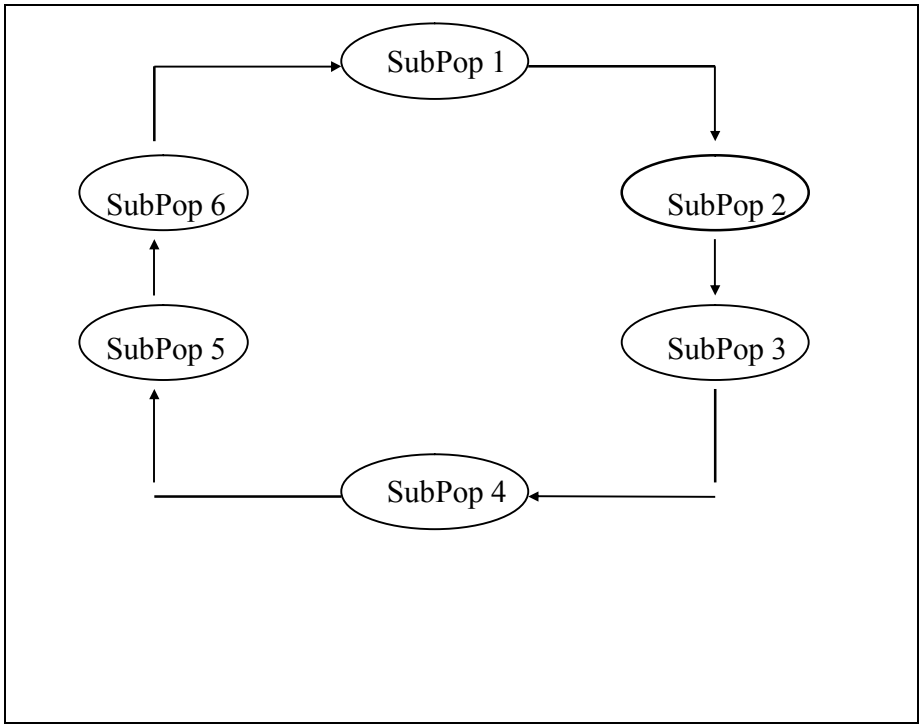
The exchange of individuals between subpopulations is called migration

Parameters:

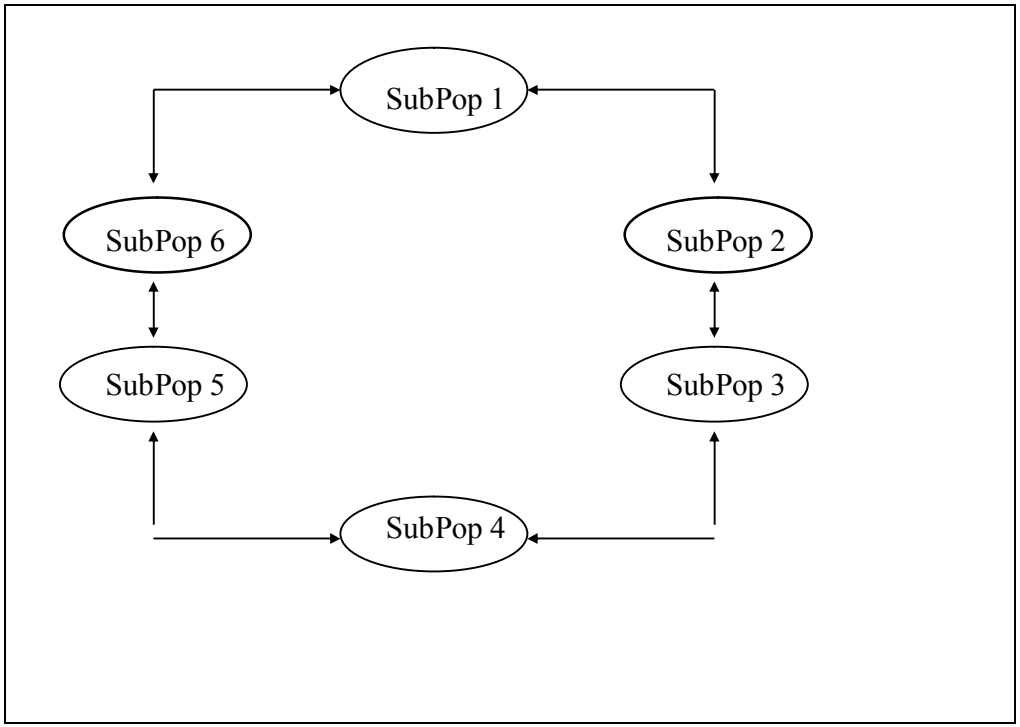
- Mihgration rate (percentage of the size of the subpopulation)
- Type of selection for migration (random or fitness based)

- migration topology:

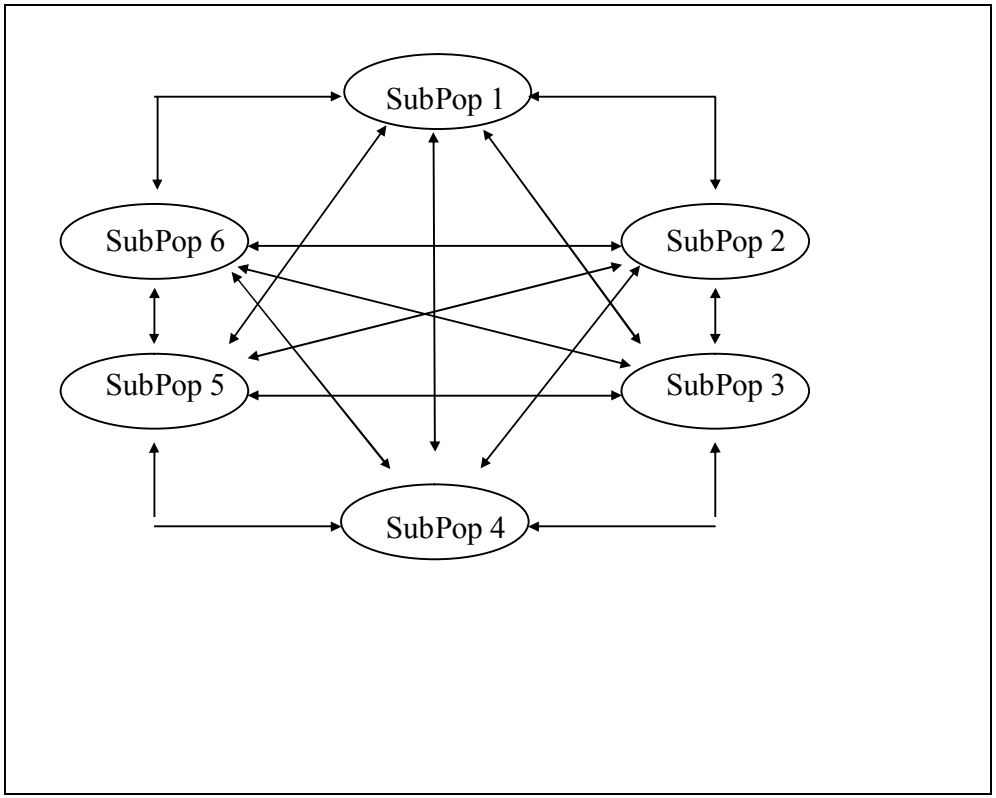
❖ Ring topology



❖ Neighborhood topology



❖ Unrestricted topology:



2.3. Testing Genetic Algorithms

- It is worth to test the efficiency and convergeny of the algorithm
- various functions (*objfun*) are used, typically with more than one local minimum

2.3.1. Typical test functions and their minimum places

2.3.1.1. De Jong's function (Global minimum: $\forall x_i = 0,\ f_1(\mathbf{0}) = 0$.)

$$f_1(\mathbf{x}) = \sum_{i=1}^{N} x_i^2, \qquad -512 \le x_i \le 512$$

Relatively simple, convex, and unimodal.

2.3.1.2. Axis-parallel hyperellipsoid (G. m.: $\forall x_i = 0,\ f_{1a}(\mathbf{0}) = 0$.)

$$f_{1a}(\mathbf{x}) = \sum_{i=1}^{N} i * x_i^2, \qquad -512 \le x_i \le 512$$

Similarly to De Jong's function, it is convex, and unimodal

2.3.1.3. Rotated hyperellipsoid (G. m.: $\forall x_i = 0,\ f_{1b}(\mathbf{0}) = 0$)

$$f_{1b}(\mathbf{x}) = \sum_{i=1}^{N} \left(\sum_{j=1}^{i} x_j \right)^2, \qquad -65 \le x_i \le 65$$

Rotating the previous function. Convex, and unimodal

2.3.1.4. Rosenbrock's banana-shaped valley (G.m.: $\forall x_i = 1,\ f_2(1,\ldots,1) = 0$)

$$f_2(\mathbf{x}) = \sum_{i=1}^{N} 100 * (x_{i+1} - x_i)^2 + (1 - x_i)^2, \qquad -2 \le x_i \le 2$$

The global optimum is inside of a long narrow parabolic valley. It is easy to find the valley but it is harder to find the minimum (not so quick convergence).

2.3.1.5. Rastrigin's function (G.m.: $\forall x_i = 0,\ f_6(\mathbf{0}) = 0$)

$$f_6(\mathbf{x}) = 10 * N + \sum_{i=1}^{N}\left(x_i^2 - 10 * \cos(2\pi x_i)\right), \quad -5.12 \le x_i \le 5.12$$

De Jong's function with cosine modulation. Many local minima. Highly unimodal. The distribution of local mimima is uniform.

2.3.1.6. Schwefel's function (G.m.: $\forall x_i = 420.9687,\ f_7(420.9687,...,420.9687) = N * 418.9829$)

$$f_7(\mathbf{x}) = -\sum_{i=1}^{N}\left(x_i * \sin\left(\sqrt{|x_i|}\right)\right), \quad -500 \le x_i \le 500$$

global minimum place is geometrically far from the next best local minimum, thus the search algorithms can potentially converge to a wrong direction

2.3.1.7. Griewangk's function (G.m.: $\forall x_i = 0,\ f_8(\mathbf{0}) = 0$)

$$f_8(\mathbf{x}) = \sum_{i=1}^{N}\frac{x_i^2}{4000} - \prod_{i=1}^{N}\cos\left(\frac{x_i}{\sqrt{i}}\right) + 1, \quad -600 \le x_i \le 600$$

Similar to Rastrigin's function

2.3.1.8. Sum of powers having different exponents (G.m.: $\forall x_i = 0,\ f_9(\mathbf{0}) = 0$)

$$f_9(\mathbf{x}) = \sum_{i=1}^{N}|x_i|^{i+1}, \quad -1 \le x_i \le 1$$

Frequently used unimodal test function.